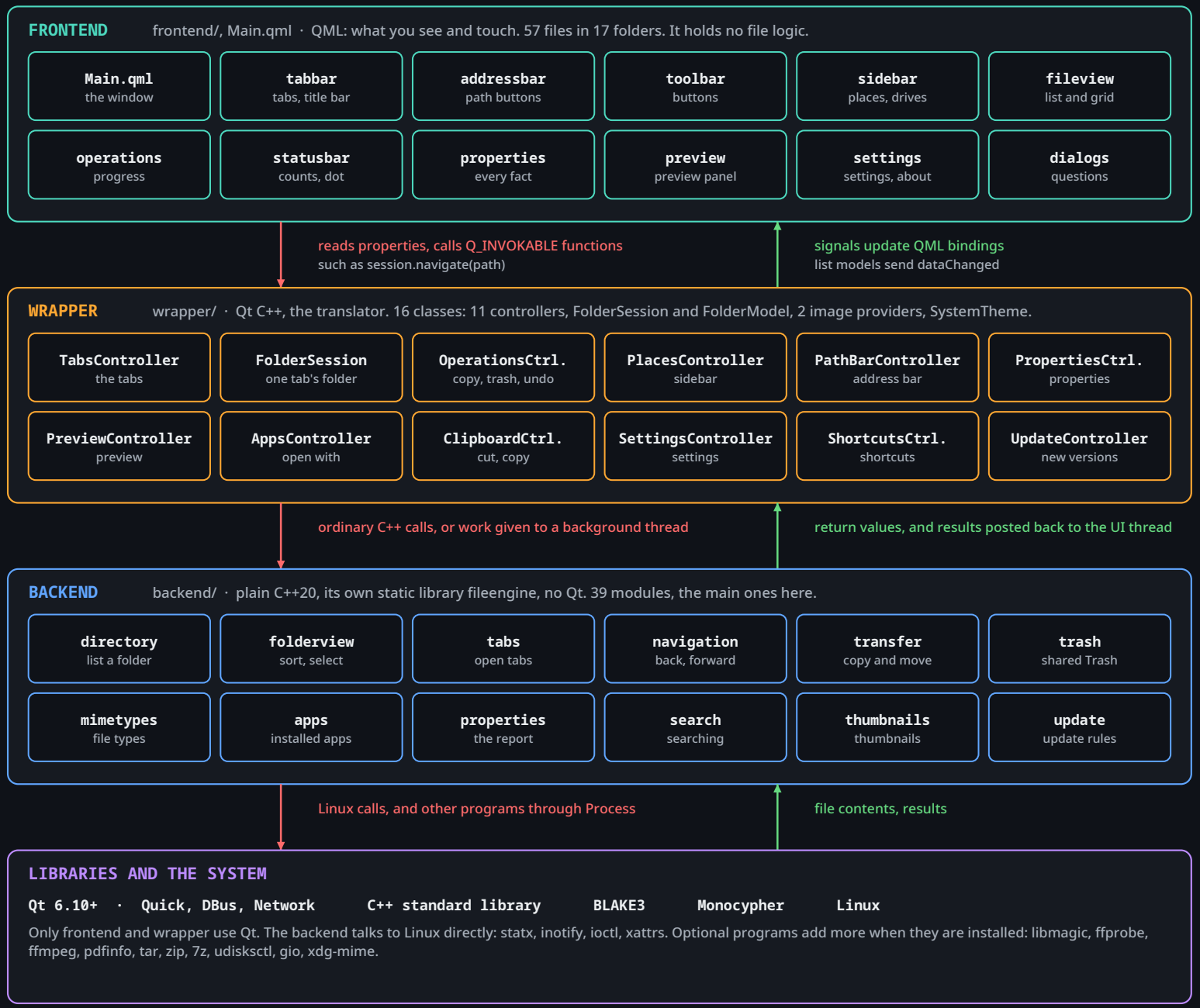
The big picture

Three layers, each talking only to the next one. The window stays on one thread, and everything slow runs on background threads.



The rules that keep it clean

The backend knows nothing about Qt.
It is a library that doesn't link Qt, so a Qt include there fails to compile.

QML never reaches the backend.
Every screen goes through a wrapper class such as TabsController.

QML has no file logic.
No paths, sorting or file work: it draws state and reports clicks.

Wrappers have no file logic either.
They connect the two sides. The rules live in the backend, where tests can reach them.

Slow work never runs on the UI thread.
A hanging disk or network share cannot freeze the window.

Colours and icons have one home.
Theme.qml holds every colour and size, Icons.qml every icon.

What is what

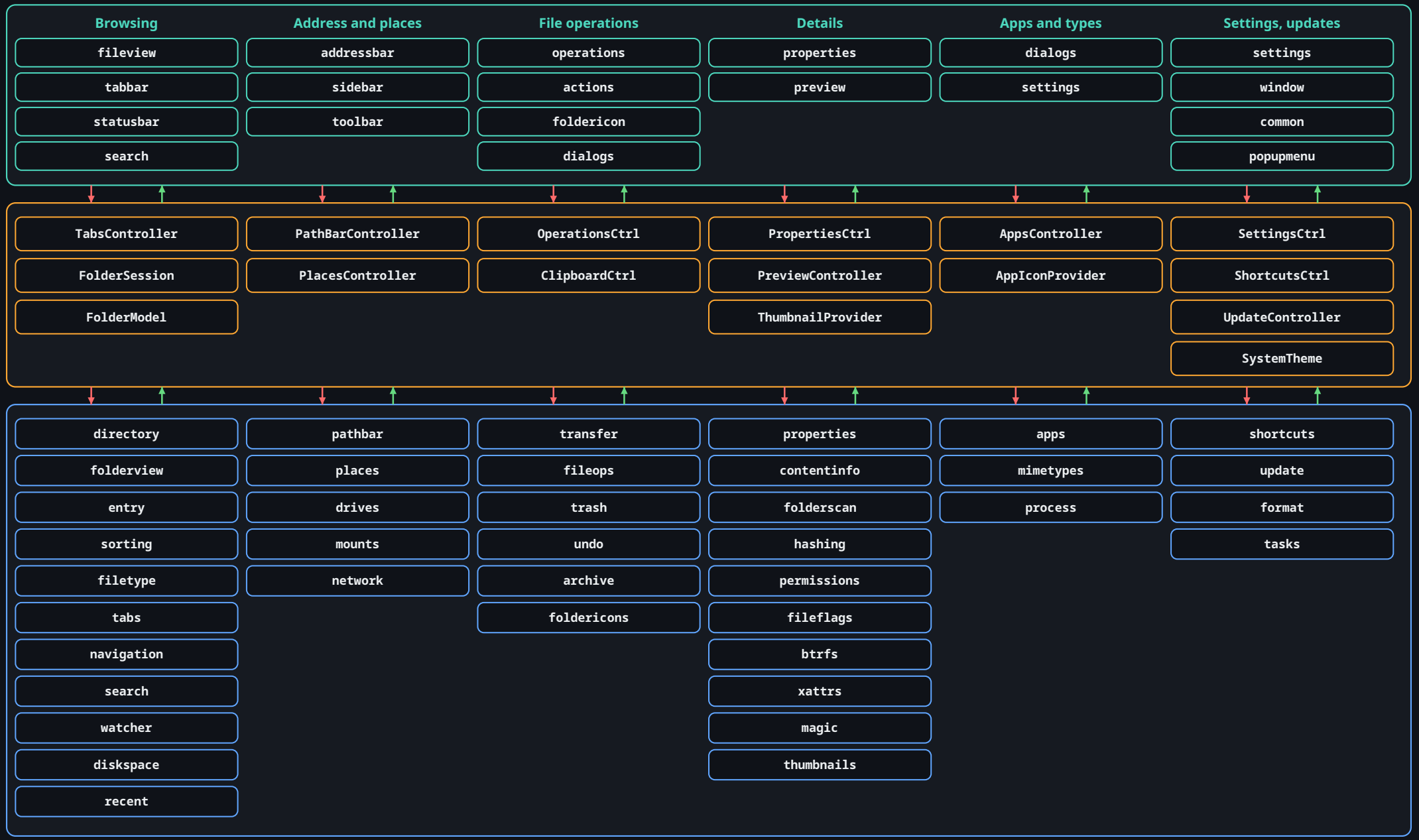
`QML`	language that describes screens
`Controller`	C++ class QML can talk to
`Singleton`	one shared object, used by name
`Signal`	a class says something changed
`Binding`	QML value that updates by itself
`Q_INVOKABLE`	C++ function QML may call
`Model`	a list QML can draw

In numbers

QML files, folders	`57, 17`
Wrapper files, classes	`34, 16`
Backend files, modules	`89, 39`
Lines: QML, wrapper, backend	`5023, 3910, 7134`
Lines of tests	`2384`

The blueprint

Read each column top to bottom: the screens, the wrapper class they use, and the backend modules behind them. All 39 modules appear.



Each module sits under the job it mainly serves, and many are shared: tasks runs background work for most controllers, and pathbar and format are used everywhere. Page 3 lists the exact links.

calls reports

The connections

Generated from the code: the QML folders that use each wrapper class, and the #include lines of each wrapper folder.

Wrapper folder	Classes	Used by these screens	Backend modules it uses	Other wrappers
appsWrap	AppIconProvider, AppsCtrl.	Main, actions, dialogs, settings	apps, entry, magic, mimetypes, search, tasks	-
clipboardWrap	ClipboardCtrl.	Main, actions	none	operations, tabs
folderWrap	FolderModel, FolderSession	by C++ only	apps, directory, filetype, foldericons, folderview, format, pathbar, recent, search, tabs, tasks, trash, watcher	apps, settings
operationsWrap	OperationsCtrl.	Main, actions, dialogs, fileview, foldericon, operations, sidebar, tabbar	archive, fileops, foldericons, format, pathbar, tasks, transfer, trash, undo	-
pathbarWrap	PathBarCtrl.	addressbar	pathbar	settings
placesWrap	PlacesCtrl.	Main, actions, dialogs, fileview, sidebar	drives, foldericons, network, pathbar, places, tasks, trash	-
previewWrap	PreviewCtrl.	preview	magic, tasks	-
propertiesWrap	PropertiesCtrl.	actions, properties	contentinfo, folderscan, format, hashing, pathbar, properties, tasks	-
settingsWrap	SettingsCtrl., SystemTheme	Main, Theme, fileview, settings, tabbar, toolbar	apps	-
shortcutsWrap	ShortcutsCtrl.	Main, actions, settings	shortcuts	-
tabsWrap	TabsCtrl.	Main, fileview, settings, sidebar, tabbar	pathbar, tabs	folder, settings
thumbnailsWrap	ThumbnailProvider	by C++ only	pathbar, thumbnails	-
updateWrap	UpdateCtrl.	settings, statusbar	hashing, process, tasks, update	operations, tabs

How to read a row: tabsWrap holds TabsController. Main, the tab bar and others use it, it uses the pathbar and tabs modules, and it reaches FolderSession and SettingsController in other wrapper folders.

FolderSession and FolderModel are not singletons. There is one of each per tab, and QML gets the current one through TabsController.currentSession, so its file shows no direct use by name. settingsWrap reaches further than its row shows: its theme mode drives Theme.qml, so every screen depends on it through colours.

thumbnailsWrap and the icon provider are not used by name either: QML reaches them through image://thumb/ and image://appicon/ addresses. fileicons in appsWrap is a helper the folder model calls for each row.

Inside the backend: which modules use which

apps	mimetypes, pathbar, process, sorting
archive	fileops, pathbar, process
btrfs	xattrs
contentinfo	apps, format, magic, process, properties
directory	diskspace, entry, filetype, pathbar, recent, sorting, trash
drives	fileops, format, process
entry	foldericons
fileops	pathbar
filetype	entry, mimetypes
foldericons	fileops
folderview	entry, search, sorting
mimetypes	pathbar
network	fileops, process
pathbar	sorting
places	pathbar
properties	btrfs, diskspace, entry, fileflags, filetype, folderscan, format, mounts, pathbar, permissions, xattrs
recent	pathbar
search	entry, filetype
sorting	entry
tabs	folderview, navigation
thumbnails	hashing, pathbar, process
transfer	fileops, pathbar
trash	fileops, mounts, pathbar

These use no other module: diskspace, fileflags, folderscan, format, hashing, magic, mounts, navigation, permissions, process, shortcuts, tasks, undo, update, watcher, xattrs.

Following one folder

You double click a folder. The work moves between two threads, so a slow disk never freezes the window.

UI THREAD

1. **FileItemArea.qml** — The row reports the double click, and FileView calls session.openRow(index). No logic in QML.
2. **openRow, navigate()** — It is a folder, so navigate() cleans the path, and history.visit() records it for Back.
3. **changeLocation()** — Emits locationChanged so the view can scroll to the top, then calls load().
4. **FolderSession::load** — Adds 1 to a generation number, shows Loading, starts the lister.

LISTER THREAD one per tab, never waited for, so a hanging network share can't freeze anything

5. **DirectoryLister** — Starting a new listing cancels the old one, so fast clicking is safe.
6. **listDirectory()** — Reads every name with statx, and asks after each one whether it was cancelled.
7. **detectFileType()** — Each entry gets its category and type name, from the name alone. No file is read.
8. **sort, disk space** — Sorts with folders first in natural order, reads free space, returns a ListResult.

UI THREAD AGAIN the result is queued back with QMetaObject::invokeMethod

9. **finishLoad()** — An old generation number means you already moved on: the result is dropped.
10. **FolderView** — setEntries stores the list between the model's begin and end signals.
11. **FolderModel** — Tells the view what changed. Each row asks for its name, type, icon and so on.
12. **FileRow / FileCell** — QML draws only what is visible. The status bar shows counts and free space.

After it opens: the folder watches itself

FolderSession starts a FolderWatcher on the folder, on its own thread, using inotify.

Bursts of changes are grouped: the callback comes when things go quiet for a moment, or after a second at the latest.

The change is queued to the UI thread. The active tab loads again from step 4. A tab in the background is only marked stale, and loads when you come back to it.

If the new listing matches what is on screen, nothing is redrawn, so your scroll position and selection stay put.

What one tab owns

Tab (backend)
a NavigationHistory and a FolderView: where it is, what it shows

FolderSession (wrapper)
that tab's lister, watcher, search job and state for QML

FolderModel (wrapper)
the rows, as a list model

TabManager (backend)
every Tab in order, the current one, and closed ones to reopen

TabsController (wrapper)
the list of sessions, and which is current

Starting up, and who runs where

main.cpp starts the app in this order. Then the threads, which are what keep a file manager smooth.

1 `restoreLibraryPath`
An AppImage's launcher set LD_LIBRARY_PATH. The loader has read it, so the user's value goes back, for every app opened later.

2 `readEnvironment`
UpdateController takes APPIMAGE and its other variables out of the environment, so they can't reach other apps.

3 `QGuiApplication`
Qt starts. Name and version are set. The desktop file name is set only if it is installed.

4 `Read the arguments`
Folders named on the command line become tabs. --tab= picks the current one, which is how an update reopens your tabs.

5 `TabsController::start`
This first use creates TabsController and SettingsController. With no folders it follows Opens to: Home, Last Tabs or a folder.

6 `QQmlApplicationEngine`
The engine starts, and the two image providers are added: thumb and appicon. Your icon theme is set here.

7 `loadFromModule`
Main.qml loads and draws the window. Other controllers are created the first time QML uses them.

8 `exec()`
The app runs until you quit. Two seconds in it reports that it started. At the end it reports that it exited.

Who runs where

Thread	How many	What it does
`UI thread`	always one	QML, all wrapper classes, models. Short work only.
`Qt render thread`	one	Qt Quick draws frames here. Not this project's code.
`DirectoryLister`	one per tab	Reads a folder. Never waited for.
`FolderWatcher`	one per tab	Waits on inotify for changes in the open folder.
`BackgroundJob: operations`	one per job	Each copy, move, trash, compress or extract. Several can run at once.
`BackgroundJob: search`	one per tab	All Subfolders search.
`BackgroundJob: properties`	two	The report and folder totals, and hashes on request. Both can be cancelled.
`BackgroundJob: preview`	one	Reads the start of a text file.
`BackgroundJob: places`	two	Scanning drives, and mounting or unmounting one.
`BackgroundJob: apps`	one	The Default Apps panel.
`BackgroundJob: update`	one	Hashing and unpacking a download.
`Thumbnail pool`	up to 4	QThreadPool: makes thumbnails, a few at a time.
`Default app queries`	8, for a moment	Asks xdg-mime for many types at once, then ends.

How the rules work

BackgroundJob never joins its thread. A stuck disk can leave a thread waiting, but the window carries on.

Cancelling sets a flag the work checks. After cancel() returns, a job can post nothing more, so a closed tab is never touched.

Results come back through a queued call, so QML and the models are only ever changed on the UI thread.

A background copy can stop to ask a question: the conflict dialog runs on the UI thread, and the copy waits on a promise for your answer.

Copying a file, and what type a file is

Two things a file manager has to get exactly right.

A copy, from paste to Undo

1 `Ctrl+V`
ClipboardController reads the clipboard, the same format other Linux file managers use.

2 `transfer()`
OperationsController adds a row to the panel and starts a BackgroundJob.

3 `Transfer::run`
Tries a btrfs reflink first, which is instant. If that fails it copies with copy_file_range.

4 `Temporary file`
It writes under a temporary name, so a failed copy leaves no half-written file.

5 `Conflict?`
The thread waits while you choose: replace, skip, keep both or cancel.

6 `Rename into place`
The finished file gets its name in one step. A replaced file stays until then.

7 `Undo step`
The result goes on UndoStack, and the message appears in the status bar.

What type is this file?

xdg-mime alone is not enough: on Hyprland it asks the file command, which calls a QML file plain text.

In a folder list: never reads the file

1. A whole name the system's list knows, such as Makefile or CMakeLists.txt.
2. The app's own table of extensions, which names the type: QML File, C++ Source.
3. The execute bit means Program.
4. The system's list, for anything else, with its own description.
5. Otherwise XYZ File, or just File.

backend/filetype and backend/mimetypes. The list is shared-mime-info: /usr/share/mime/globs2. It is read once into hash tables, so a huge folder stays fast.

To open it: name first, content second

1. Folders always go to xdg-mime.
2. One candidate from the name: that is the type. main.cpp is text/x-c++src.
3. Several candidates, such as .ts: the content decides. Text means TypeScript, not video.
4. No candidate: xdg-mime reads the content.
5. The default app comes from the type, then what it is a kind of: QML, then plain text.

DesktopApps::openingType and defaultFor. Open With and the Default Apps panel go by the same types.

Its icon

1. Folders use the app's own icon, and can have a chosen one and a colour.
2. Images, video and PDF get a thumbnail.
3. Other files: the system's list gives icon names, such as text-x-qml.
4. The first name your icon theme has is used, cached by type.
5. No icon in the theme: Icons.qml draws one by category.

FileIcons in appsWrap, and TypeIcon.qml. The theme comes from Qt, or from the GTK settings on Hyprland.

Building, releasing, updating

From source to a running app, and how an installed copy replaces itself.

Build

`blake3, monocypher`	third-party C code, as static libraries
`fileengine`	every backend file: no Qt
`appChaoticFiles`	main.cpp, wrapper, QML module, the font
`backendtests`	ctest: the engine with no window
`tests/update`	the update system end to end, by hand

A release: only a v* tag starts it, on GitHub, on Ubuntu 22.04

`checkout` → `Qt 6.11.2` → `build` → `tests` → `sign` → `smoke test` → `publish`

packaging/build-appimage.sh makes the AppImage and tarball. packaging/sign-update.sh writes update-manifest.txt and signs it. It stops if the secret key doesn't match packaging/update-public-key.txt, and a tag with no key publishes nothing. The smoke test is advisory: a failure there does not stop a release.

An update, step by step

Nothing updates by itself. Every step that can go wrong leaves the old version working.

1 Check
Once a day if the switch is on: asks GitHub for the latest tag. Offline stays silent. A newer one shows the green dot.

2 Verify
Update now fetches the manifest and signature first. Checks Ed25519 with the built-in key, then the version.

3 Download
Next to the program, so the swap is a plain rename. Checks the SHA-256 from the manifest, then unpacks.

4 Ask
Other window open: stops. Copies running: waits. Cancel leaves every later quit a normal one.

5 Swap
The app quits and a script takes over. It waits for the old-exited file, then renames the copies.

6 Confirm
The new version writes a started file after 2 seconds. The script then deletes the backup.

What stops a bad update

Wrong key, edited manifest
The signature fails: the file is not even downloaded.

Changed file
Its hash no longer matches the signed one.

An old release
The version must be newer than the running one.

Tricky file names
The manifest's names must be exactly what the version gives.

A stalled server
Requests give up after 30 seconds.

When the new version fails

The script gives it one minute to write the started file. It watches for files, because a crashed process can still look alive as a zombie.

If it exits, crashes or hangs, the script stops it, puts the old copy back and starts it with your tabs. It leaves a .failed mark, so the old version can show a notice.

A second window started during the swap sees the installing mark and leaves the update folder alone.

Who can update, and where the key is

Tarball
~/.local/lib/ChaoticFiles: the folder is swapped. Your data in ~/.local/share and settings in ~/.config stay.

AppImage
The file itself is swapped, if its folder can be written.

sudo ./install.sh, read-only copies
Only the dot and a link to the release page.

The key
The private key is only in GitHub secrets and your KEYS folder. The app holds the public key.